

Verification Substrates Inherit Paper 1's Conflict Preservation

Series C Derivation Note C1.24 — #453

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 14, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work formalizes an inheritance edge between Paper 2 (Li, April 2026) and Paper 1 (Li, April 2026) of the Coordination Knowledge Substrate (CKS) theory series.

Abstract

Paper 2 of the Coordination Knowledge Substrate (CKS) theory series introduces verification substrates: DNA-layer objects that specify a parallel-run mechanism, submitting the same input to two execution surfaces and comparing the results. When the two surfaces diverge, a behavioral deviation is detected. This note establishes that verification substrates inherit Paper 1's conflict preservation commitment — that when two governed specifications produce different results for the same operational territory, both sides must be preserved as substrate content, registered as a first-class conflict object, and resolved through governance rather than automated collapse. The inheritance is direct and precise: a behavioral deviation detected by a verification substrate is a conflict in the Paper 1 sense, and all four requirements of Paper 1's conflict preservation commitment apply without modification. What Paper 2 contributes is not a new conflict concept but a specific, governed detection mechanism — the parallel-run as authored substrate content — and the mutation governance use case that makes the mechanism architecturally significant. This note also positions C1.24 as the complement to C1.17 (instinct evolution $\supset$ tool-agnosticism), which together cover the full mutation governance inheritance territory: C1.17 establishes why mutation is architecturally expected; C1.24 establishes how its behavioral effects are detected and governed.

1. The inheritance edge

C1.24 establishes the following inheritance relationship:

Paper 2 verification substrates $\supset$ Paper 1 conflict preservation (parallel-run as detection mechanism)

The symbol $\supset$ denotes proper extension: Paper 2's verification substrates carry forward every architectural commitment Paper 1's conflict preservation requires, and add new content at the detection mechanism and object-type levels that Paper 1 did not specify.

The inherited side is Paper 1’s Claim 2, formalized as A0.02 in the derivation note series: conflict preservation as a first-class architectural property. When two governed specifications produce conflicting states for the same operational territory, both sides are preserved in the substrate; conflicts are not auto-resolved; they are registered, attributed, and governed through resolution rules.

The extending side is Paper 2’s verification-substrate machinery: the parallel-run pattern that detects behavioral deviations when an execution surface changes, authors the detection specification as DNA-layer substrate content, and triggers a governance checkpoint when a deviation is found.

2. Paper 1’s conflict preservation commitment (the inherited side)

Paper 1 commits to conflict preservation as an architectural property at four levels.

Substrate-level preservation.

When two governed specifications produce conflicting states, both sides are preserved as live substrate content. The substrate does not collapse the conflict into a single winner and discard the other. Both results remain addressable. This is the substrate-level half of the two-level commitment: preservation at the data layer, not resolution.

First-class registration.

A preserved conflict is not merely the co-residence of two inconsistent pieces of substrate content. It is a first-class substrate object with its own addressable identity: which content is in conflict, on what dimension the conflict holds, what the provenance of each side is, and when the conflict was registered. The conflict relationship is substrate state, not a condition a reader must reconstruct by inspecting both sides independently.

Governed resolution.

Cell-level response to a preserved conflict follows human-authored orchestration rules. No LLM operation, automated process, or runtime middleware layer can silently collapse a conflict. Collapse requires either direct human action or a human-authored orchestration rule that explicitly authorizes it for the specific case. Human deferral — choosing not to resolve a conflict at the moment it surfaces — is a recorded terminal outcome, not a failure mode.

Detection as prerequisite.

Paper 1’s conflict preservation commitment implicitly requires that conflicts be detected before they can be preserved. Two specifications must be identified as covering the same operational territory and producing different outputs before the preservation machinery can register the conflict. Paper 1 commits to this requirement without specifying how detection occurs.

These four commitments together are what Paper 2’s verification substrates must carry forward to qualify as inheriting Paper 1’s conflict preservation. All four are carried forward, as §4 establishes.

3. Paper 2’s verification substrates (the extending side)

Paper 2 introduces verification substrates as a DNA-layer object type governing instinct integration. When instinct sharpens — most commonly when a new execution surface version becomes available upstream — the system must determine whether the new version produces behavioral outputs that are consistent with the previous version or whether it introduces behavioral deviations.

The parallel-run mechanism.

A verification substrate specifies that the same input be submitted to two execution surfaces in parallel. The current surface (the established version) and the candidate surface (the new or modified version) both receive the same substrate state as input. Their outputs are compared. If the outputs agree within the deviation tolerance the verification substrate specifies, the candidate surface passes. If the outputs diverge beyond that tolerance, a behavioral deviation is detected.

Verification substrates as authored substrate content.

The specification for what to run in parallel, what inputs to use, and what constitutes an acceptable deviation is itself DNA-layer substrate content. It is human-authored in the same sense all DNA content is human-authored: authored by humans or by LLMs under human direction, subject to the three governance rights (inspect, modify, override), and changeable under the same authority architecture that governs all DNA content. The verification machinery is not hardwired into the system's runtime; it is authored. Governance can change what gets verified, what deviation tolerance applies, and when a prior execution surface may be retired — and those changes take effect as substrate state, not as configuration parameter adjustments made outside the governance boundary.

Deviations as governance events.

When a verification substrate detects a deviation, the detection event triggers a governance checkpoint: a verification gate at which human governance determines whether the deviation is acceptable (the new execution surface passes verification) or problematic (it fails, and the prior surface remains active). The gate structure is a governance checkpoint, not an automated decision. It is one instance of Paper 2's broader commitment to multi-shaped governance: authority over instinct integration residing with humans, exercised at the verification gate, rather than delegated to automated comparison logic.

4. The inheritance relationship: what conflict preservation properties are carried forward

The claim of the C1.24 inheritance edge is that a behavioral deviation detected by a verification substrate is a conflict in the Paper 1 sense, and that all four requirements of Paper 1's conflict preservation commitment apply without modification.

Why a deviation is a Paper 1 conflict.

Paper 1 defines a conflict as the condition that arises when two governed specifications cover the same operational territory and produce different outputs. A verification substrate creates exactly this condition, in a structured and authored form: two execution surfaces operate on the same substrate state (same input, same operational territory); they produce different outputs (a deviation); both surfaces are

governed — the current surface through the orchestration rules that put it in service, the candidate surface through the verification substrate that authorizes its parallel execution. The three conditions for a Paper 1 conflict are all present. No new conflict concept is required; the existing definition applies directly.

Both sides preserved.

When a verification substrate detects a deviation, both execution results — the current surface’s output and the candidate surface’s output — are preserved as substrate content. Neither is discarded. The preservation is direct inheritance of Paper 1’s substrate-level preservation requirement.

First-class registration.

The detected deviation is registered as a first-class conflict object in the substrate: which execution surfaces are in conflict, on what input the deviation was detected, what the outputs were, when the detection event occurred, and what the verification gate determination was (or that determination is pending). The conflict relationship is substrate state with its own addressable identity — directly inheriting Paper 1’s first-class registration requirement.

Not auto-resolved.

The verification gate is a governance checkpoint, not an automated resolver. The parallel-run mechanism detects and preserves the conflict; it does not automatically determine which execution result is correct or authorize the candidate surface to replace the current one. Human governance makes the determination. This directly inherits Paper 1’s governed-resolution requirement.

Detection before preservation.

The verification substrate is the detection mechanism. The parallel-run executes before preservation: two surfaces produce their outputs, the outputs are compared, a deviation is identified as such, and then the preservation and registration machinery operates on the identified conflict. This satisfies Paper 1’s implicit detection-as-prerequisite requirement — and Paper 2 makes the satisfaction explicit by specifying the detection mechanism, which Paper 1 left open.

5. What is new in Paper 2

The inheritance is genuine: Paper 2’s verification substrates do not introduce a new conflict concept. But three elements are new in Paper 2, and naming them precisely is what prevents the inheritance from collapsing into mere re-citation.

Parallel-run as the explicit detection mechanism.

Paper 1 committed to conflict detection as a prerequisite for conflict preservation but did not specify how detection occurs. Paper 2 introduces the parallel-run as the explicit mechanism: running two execution surfaces on identical input and comparing outputs is the canonical method for detecting behavioral conflicts at execution-surface boundaries. The mechanism is specific, operationally concrete, and authored as substrate content rather than embedded in runtime comparison logic.

Verification substrates as a DNA-layer object type.

The parallel-run configuration is a new object type in the substrate architecture — a DNA-layer substrate that specifies detection behavior rather than coordination behavior. This object type is governed the same way all DNA content is governed: it is authored, subject to the three governance rights, and changeable under human authority. Introducing the object type is fresh Paper 2 content that Paper 1 does not contain.

Mutation governance as the primary use case.

The principal application of verification substrates is governing instinct evolution: detecting whether a new execution surface version produces behavioral deviations from the established version. This use case — conflict detection at execution-surface version boundaries, with deviation detection triggering a verification gate — is architecturally significant in Paper 2 and does not appear in Paper 1. Paper 1's conflict preservation was formalized at the level of governed specification conflicts within a cell; Paper 2 extends the same machinery to the execution-surface upgrade event.

6. Relationship to C1.17: the complete mutation governance picture

C1.24 is the complement to C1.17 (Paper 2 instinct evolution $\supset$ Paper 1 tool-agnosticism). Together the two notes cover the full inheritance territory for mutation governance.

C1.17 established that instinct evolution inherits Paper 1's tool-agnosticism: because the execution surface and infrastructure occupy the host layer, changes to execution surface versions are architecturally expected events rather than exceptional disruptions. Paper 1's tool-agnosticism commits to the architecture remaining stable across host-layer changes; Paper 2's instinct evolution inherits that commitment by treating version changes as a governed evolution mechanism rather than a threat to architectural invariants.

C1.24 establishes how the behavioral effects of those version changes are detected and governed. Once a version change arrives, the architecture needs a mechanism for determining whether the change produces behavioral deviations — and if it does, for preserving those deviations as governed conflict objects rather than silently absorbing them. Verification substrates are that mechanism, and their conflict-preservation behavior is inherited from Paper 1.

The two notes are not redundant. C1.17 answers: why is mutation architecturally expected, and why does it not threaten Paper 1's commitments? C1.24 answers: when mutation occurs, how does the architecture detect and govern its behavioral effects? A reader who has seen only C1.17 has the conceptual framing for mutation governance but not the detection mechanism. A reader who has seen only C1.24 has the detection mechanism but not the reason mutation events arise in the first place. Together they provide the complete picture.

7. Prior-art significance

C1.24 forecloses three classes of adversarial claim.

First class. A claim that parallel-run behavioral comparison as a conflict detection mechanism is novel relative to Paper 1's conflict preservation. C1.24 establishes that parallel-run is the explicit detection mechanism for a conflict type that Paper 1 already committed to preserving. The detection mechanism is new in Paper 2; the conflict preservation requirement it serves is not. An adversarial claim that the combination of parallel-run detection and conflict preservation constitutes a novel conflict-handling architecture misreads the relationship: Paper 2 fills in the detection mechanism Paper 1 left unspecified; it does not introduce a new conflict concept.

Second class. A claim that treating execution-surface behavioral deviations as conflicts requiring preservation and governance is novel. C1.24 establishes that the conflict-preservation requirements for detected deviations follow directly from Paper 1: both sides preserved, first-class registration, governed resolution. The application to execution-surface version boundaries is new in Paper 2 (the mutation governance use case), but the preservation requirements that application triggers are inherited from Paper 1.

Third class. A claim that verification substrates as authored DNA-layer objects specifying parallel-run configurations are novel as a combined architectural element. C1.24 establishes that the DNA-layer object type is new in Paper 2 but the conflict-preservation behavior it produces is inherited from Paper 1. The substrate-content character of the verification specification — the fact that the parallel-run configuration is authored governance content rather than hardwired runtime logic — is a direct application of Paper 1's commitment to orchestration rules as human-authored substrate content, extended to verification specifications.

8. Operational test

A system implements the C1.24 inheritance relationship if and only if all of the following are true whenever a verification substrate detects a behavioral deviation:

1. **Both execution results are preserved as substrate content.** The current execution surface's output and the candidate surface's output are both recorded in the substrate and remain addressable after the detection event. Neither is discarded when the deviation is detected.
2. **The deviation is registered as a first-class conflict object.** The conflict has its own addressable substrate identity: which execution surfaces are in conflict, what input produced the deviation, what the outputs were, when detection occurred, and attribution for the detection event. The conflict relationship is substrate state, not a property reconstructed by comparing two stored outputs.
3. **Governance determines the outcome; the parallel-run does not.** The parallel-run mechanism detects and preserves the conflict. It does not determine which execution result is correct or authorize the candidate surface to supersede the current one. A governance determination — accepting or rejecting the candidate surface — is required and is itself recorded as substrate content with its own writer, timestamp, and reference to the verification gate event that prompted it.

4. **The verification substrate specification is authored substrate content.** The specification for what to run in parallel, what input to use, and what constitutes an acceptable deviation is DNA-layer substrate content subject to the three governance rights (inspect, modify, override). It is not hardwired runtime comparison logic. Governance can change the specification; the change takes effect as substrate state under the same authority architecture that governs all DNA content.
5. **The detection event is attributed and reachable.** An observer starting from the registered conflict object can trace to the verification substrate that specified the detection, the governance determination that resolved or deferred it, and the execution surfaces that were compared. The path is reconstructable from substrate content without consulting runtime logs outside the substrate.

A system that satisfies (1)–(3) but fails (4) has implemented conflict preservation but not the verification-substrate object type Paper 2 introduces. A system that satisfies (4) but fails (1)–(3) has authored a parallel-run specification but has not implemented the conflict preservation that Paper 1 requires when the parallel-run detects a deviation. Full implementation of the C1.24 inheritance relationship requires all five conditions.

9. Conclusion

Paper 2’s verification substrates inherit Paper 1’s conflict preservation commitment at four levels: both sides of a detected deviation are preserved as substrate content, the deviation is registered as a first-class conflict object, resolution is governed rather than automatic, and the detection mechanism satisfies the detection-as-prerequisite requirement Paper 1 implies. The inheritance is direct — a behavioral deviation produced by a parallel-run is a conflict in the Paper 1 sense, and no new conflict concept is required.

Paper 2 adds three genuinely new elements on top of the inheritance: the parallel-run as the explicit detection mechanism, verification substrates as a DNA-layer authored object type, and the mutation governance use case that makes the mechanism architecturally significant. These additions extend Paper 1’s conflict preservation into the execution-surface upgrade event without displacing the inherited commitments.

Read together with C1.17, this note completes the mutation governance inheritance picture: C1.17 explains why execution-surface changes are architecturally expected; C1.24 explains how their behavioral effects are detected as governed conflicts. Both notes are required to understand how the CKS architecture treats instinct evolution as a governed process rather than an uncontrolled capability change.

Source papers

Li, W. (2026a). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026b). [*Paper 2: CKS Series, April 2026*]. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Verification Substrates Inherit Paper 1's Conflict Preservation: Series C Derivation Note C1.24*. May 14, 2026. ORCID: 0009-0004-8065-3235.